Unit - 7

Pipeline Processing [10%]

1. Parallel Processing :-

→ It is a term used to denote the technique of simultaneous data processing tasks for increasing the computational speed of computer systems.

→ Instead of processing each instruction sequentially as in conventional system, a parallel processing system is able to perform data processing faster & short execution time.

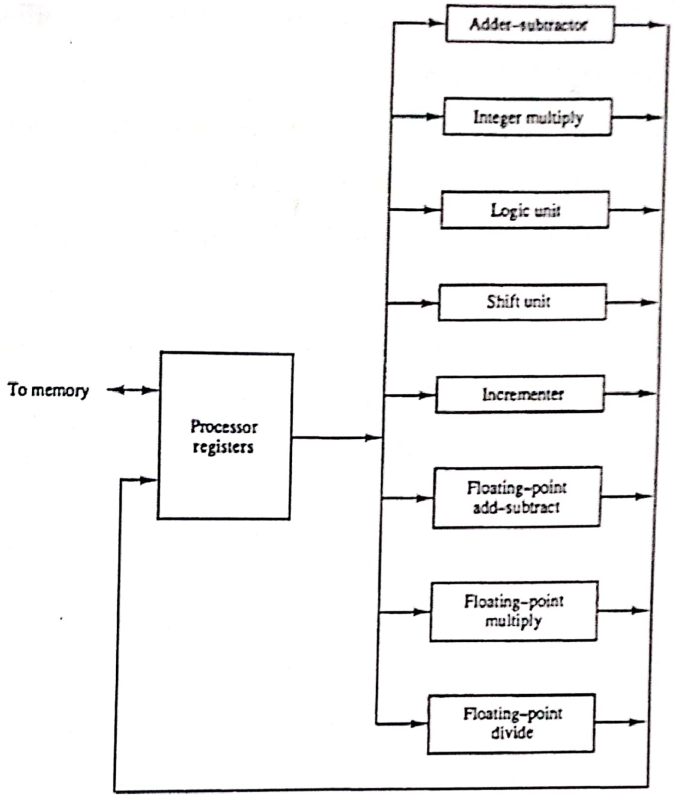
→ For example, while an instruction is being executed in ALU, the next instruction can be read from memory.

→ The system may have two or more ALU's and be able to execute two or more instructions at the same time.

→ Purpose of parallel processing is to speed up the computer processing capability & increase its throughput.

→ Parallel processing can be viewed from various levels of complexity.

→ Fig-1 shows one possible way of seperating execution unit into 8 functional units operating in parallel.



→ There are Variety of ways that parallel processing can be classified.

→ One classification is introduced by M. J. Flynn.

→ The normal operation of computer system is to fetch instructions from memory & execute them in processor.

→ Flynn's classification divides computers into 4 major groups as follows:

1. SISD (Single Instruction Single Data)
2. SIMD (Single Instruction Multiple data)
3. MISD (Multiple Instruction Single data)
4. MIMD (Multiple Instruction Multiple data)

→ SISD represents Single computer with Control Unit, processor Unit & memory Unit. Instructions are executed sequentially

→ SIMD represents Computer System with many processing unit and Common Control Unit. All processors receive the same instructions from control unit but operate on different datas.

→ MISD Structure is only of theoretical interest since no practical system has been constructed using it.

→ MIMD refers to a computer system capable of processing several programs at the same time. Most multiprocessor & multi-computers systems are of this type

→ Flynn's classification depends on performance of Control Unit & data processing unit.

* Parallel Processing
  - → Pipeline Processing
  - → Vector Processing
  - → Array processors

⇒ Pipelining :-

→ It is a technique of decomposing sequential process into sub-operations.

→ Each subprocess being executed in special dedicated segment.

→ The idea behind pipelining concept is to process more than one instruction by the processor at a same time.

→ Pipelining is the process of arrangement of hardware elements of CPU such that its overall performance is increased.

→ Let's see real life example to understand the concept of pipelining :-

* Consider Water bottle Packaging Plant :

→ 3 Stages from which bottle should pass :

Stage 1 : Inserting the bottle (I)
Stage 2 : Filling Water (F)
Stage 3 : Sealing the bottle (S)

* In non-pipelined structure :

→ Let each stage take 1 min to complete its operation.

→ Bottle is first inserted (I).
→ After 1 min it is moved to Stage-2 where bottle is filled. (F)
→ When operation is in Stage-2 is on going at the same time nothing is happened in Stage-1.
→ Similarly when bottle moves to Stage-3 both Stage-1 & 2 become idle.

* In pipelined structure :-

→ When the bottle is in 2nd Stage, another bottle can be loaded at Stage-1.

→ Similarly when the bottle is loaded into 3rd Stage, there can be one bottle in stage-1 and 2.

⇒ Without pipelining:

```
I F S - - - - - -
- - - I F S - - -
- - - - - - I F S
```

Average time taken to manufacture 1 bottle : $\frac{9}{3}$ = 3 min

⇒ With Pipelining:

```
I F S - -
- I F S -
- - I F S
```

Avg. time = $\frac{5}{3}$

= 1.67 min

→ The pipeline organization will be explained by following example :

Suppose that we want to perform combined multiply & add operations with stream of no :

$$A_i * B_i + C_i \quad \text{for } i = 1, 2, 3 \ldots 7$$

→ Each suboperation is to be implemented in a segment within a pipeline.

→ Each segment has 2 registers. R1 to R5 are registers that receive new data with every clock pulse.

→ The suboperation performed in each segment of pipeline are as follows :

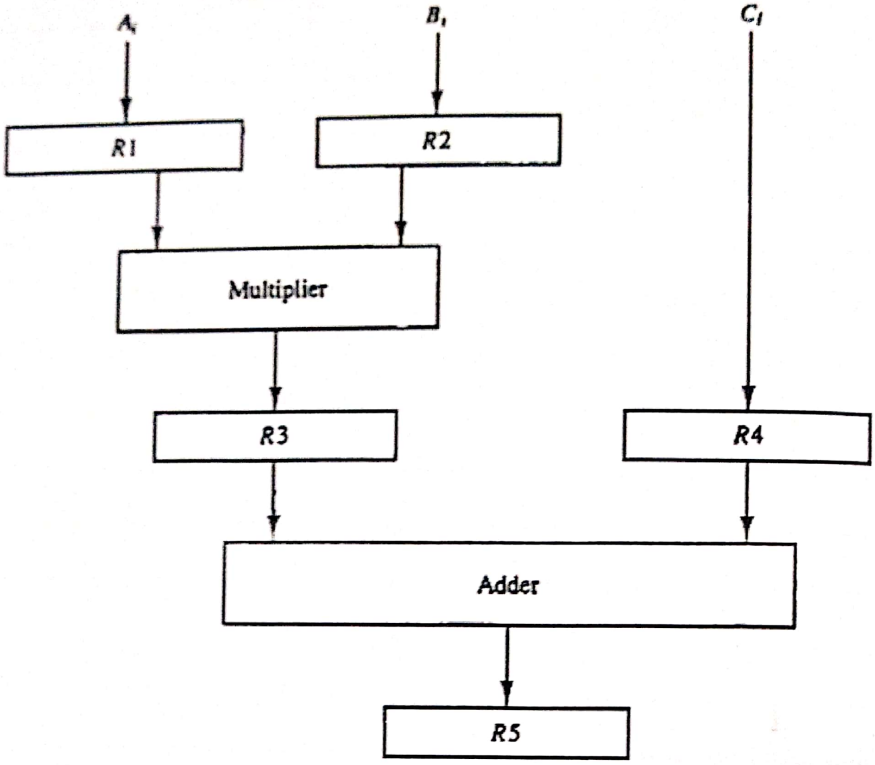
$R_1 \leftarrow A_i$, $R2 \leftarrow B_i$ (Input A_i & B_i)

$R3 \leftarrow R1 * R2$, $R4 \leftarrow C_i$ (Mul & i/p C_i)

$R5 \leftarrow R3 + R4$ (Add C_i to product).

→ The 5 registers are loaded with new data every clock pulse.

→ The effect of each clock is shown in table - 1.



Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

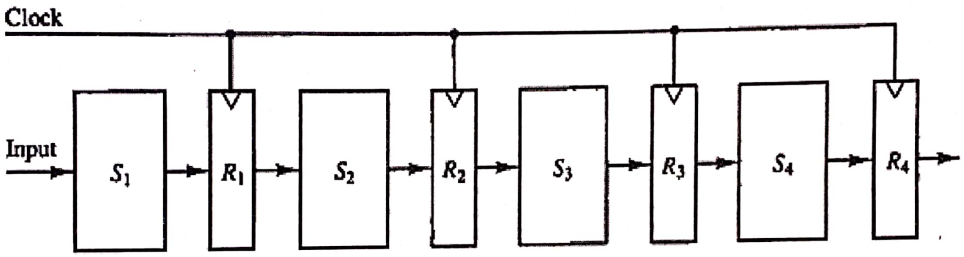
The first clock pulse transfers A_i & B_i into R1 and R2

Second clock pulse transfers product of R1 & R2 into R3 and C_i into R4.

→ The same clock pulse transfers A_2 and B_2 into R1 and R2.

→ Similarly, the 3rd clock pulse operates on all 3 segments simultaneously.

→ This technique is efficient for those applications that need to repeat the same task many times with different set of data.



→ The operands pass through all 4-segment in a fixed sequence.

→ Each segment consists of combinational ckts S_i that perform sub-operation

→ The segments are seperated by registers R_i that hold the intermediate results between the stages.

→ Task is defined as total operation performed going through all the segments in the pipeline.

Space - Time Diagram :

→ The behaviour of pipeline can be illustrated with space - time diagram.

→ It is the diagram that shows segment utilization as a function of time.

→ The space - time diagram for four segments pipeline is shown in fig.

→ Horizontal axis : Clock cycles
Vertical axis : Segment Numbers

→ Fig shows 6 tasks : T_1 to T_6 executed in 4 segments.

→ Task - T1 is handled by Segment - 1. After the 1st clock, segment - 2 is busy with T_1, while segment - 1 is busy with task T_2.

→ So, first task is completed with 4th clock cycle.

Segment:	1	2	3	4	5	6	7	8	9	→ Clock cycles
1	T_1	T_2	T_3	T_4	T_5	T_6				
2		T_1	T_2	T_3	T_4	T_5	T_6			
3			T_1	T_2	T_3	T_4	T_5	T_6		
4				T_1	T_2	T_3	T_4	T_5	T_6	

→ Now consider the case where k-segment pipeline with clock cycle time t_p is used to execute 'n' tasks.

→ 1st task T1 requires a time equal to kt_p to complete its operation.

→ Remaining n-1 tasks emerge from the pipe at the rate of 1 task per clock cycle & they will be completed after a time equal to $(n-1)t_p$

→ To complete n tasks using k segment pipeline requires: $k + (n-1)$ clock cycle.

→ For example, diagram shows:

No. of segment = 4

Tasks = 6

Time required to complete all operations = $4 + (6-1) = 9$

= 9 clock cycles

→ Now, consider non-pipeline unit that performs same operation at time equal to t_n to complete each task.

Total time required for 'n' tasks
= $n t_n$

Speed up Ratio

→ The speedup of a pipeline measures how much more quickly a task is completed by the pipeline processor than by non-pipeline processor.

$$S = \frac{\text{time taken in non-pipeline system}}{\text{time taken in pipeline system}}$$

$$S = \frac{n t_n}{(k + n - 1) t_p}$$

Where k = no. of segment
n = no. of tasks

As the no. of task increases, n becomes much larger than $k-1$ $(k + n - 1)$ tends to the value of 'n'. Under this condition, the speed-up becomes,

$$S = \frac{t_n}{t_p}$$

If we assume that, time it takes to process a task is the same in the pipeline & non-pipeline circuits we will have $t_n = k t_p$

So, $S = \frac{k t_p}{t_p} = k$

So, Maximum Speedup = S = No. of segment
= k

⇒ Consider the following numerical example :

→ Time required to process a task in each segment = t_p = 20 ns

→ Assume that pipeline has k = 4 segments
Total tasks = n = 100 tasks

→ Pipeline system will have to take,
$(k + n - 1) t_p = (4 + 99) \times 20 = 2060$ ns
= 2060 ns is required to complete.

→ Assume that $t_n = k t_p = 4 \times 20 = 80$ ns

→ Non-pipeline system requires $n k t_p = n t_n$
$= 100 \times 80$
$= 8000$ ns

→ Non-pipeline system needs 8000 ns to complete 100 tasks.

→ Pipeline system needs only 2060 ns to complete 100 tasks.

Speed up Ratio = $S_. = \frac{8000}{2060} = 3.88$

→ As the tasks increases, Speed-up will approach 4, which is equal to no. of segment in pipeline.

→ There are two areas of computers design where pipeline structure is applicable.

1. Arithmatic Pipeline
2. Instruction Pipeline

* Arithmatic Pipeline :-

→ Pipeline arithmatic units are found in very high speed computers.

→ They are used to :
1. implement floating point operations
2. Multiplication of fixed point numbers
3. implement scientific problems.

→ Consider an example of pipeline unit for floating point addition & subtraction

→ The inputs to floating point adder pipeline are two normalized floating point binary no:

$$X = A \times 2^a$$
$$Y = B \times 2^b$$

→ A and B are two fractions that represents mantissas and a, b are exponents.

→ The registers labeled R are placed between the segments to store intermediate results.

→ The sub-operations that are performed in four segments are:

1. Compare the exponent
2. Align the mantissas
3. Add or subtract the mantissas
4. Normalize the result.

→ The exponents are compared by subtracting them to find their difference.

→ The two mantissas are added or subtracted in segment - 3.

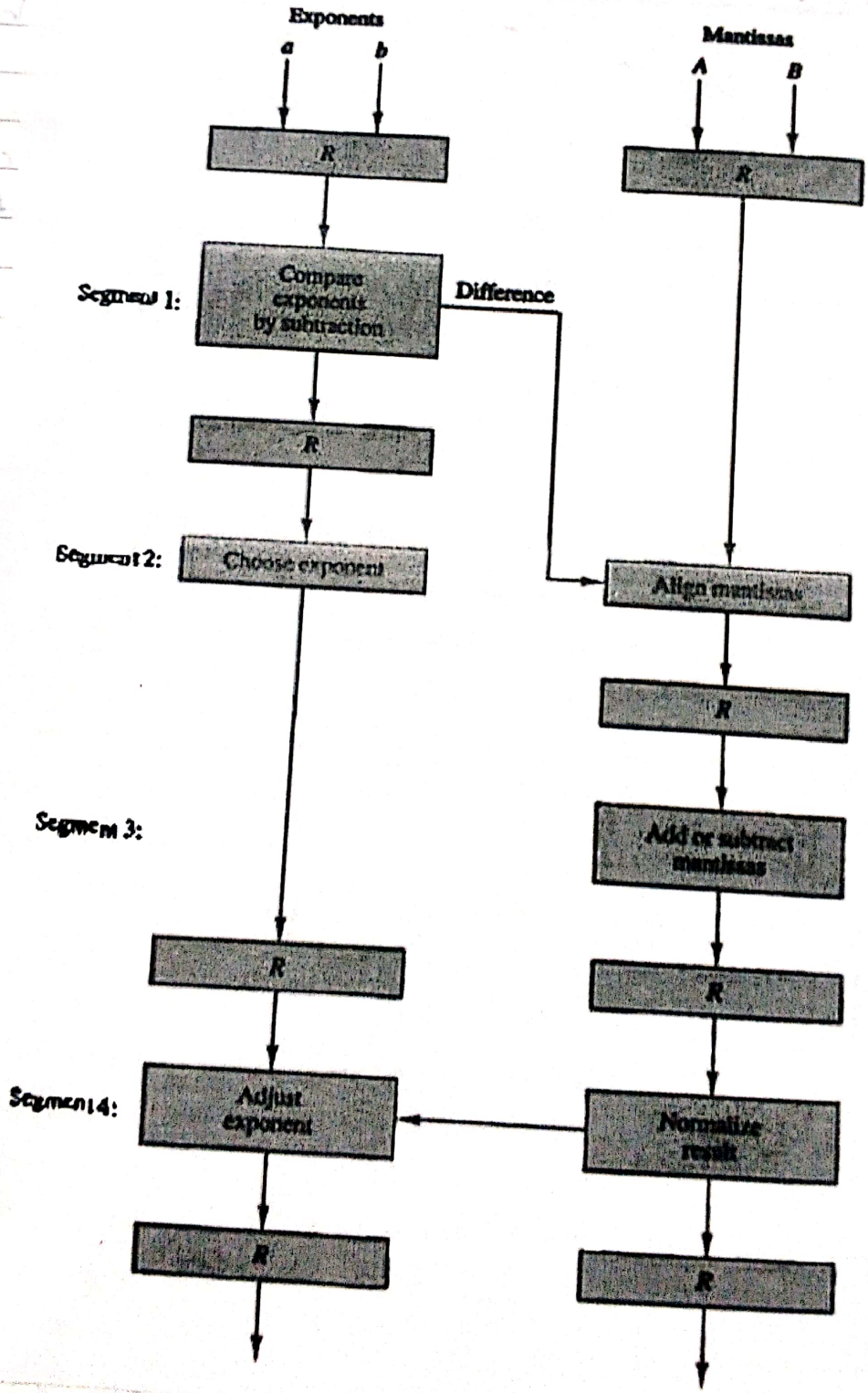
→ The result is normalized in segment - 4.

→ The following numerical may clarify the suboperations performed in each segment.

→ Consider the two normalized floating point numbers:

$X = 0.9504 \times 10^3$

$Y = 0.8200 \times 10^2$



[Pipeline for floating point addition or Subtraction]

⇒ Compare the exponents :-

→ The two exponents are subtracted in the first segment to obtain : 3-2 = 1

→ The larger exponent 3 is chosen as the exponent of result.

→ The next segment shifts the mantissa of Y to the right to obtain

$$X = 0.9504 \times 10^3$$

$$Y = 0.0820 \times 10^3$$

→ This aligns the two mantissas under the same exponent.

→ The addition of two mantissas in segment 3 produces the sum :

$$Z = 1.0324 \times 10^3$$

→ The sum is adjusted by normalizing the result so that it has a fraction with non-zero first digit.

$$Z = 0.10324 \times 10^4$$

→ Components used in arithmatic pipeline :

- → Comparator
- → Shifters
- → Adder - Subtractor
- → Incrementers
- → Decrementers

→ Suppose that time delay of 4-segments are:

$t_1 = 60$ ns
$t_2 = 70$ ns
$t_3 = 100$ ns
$t_4 = 80$ ns
interface registers delay $= t_r = 10$ ns

Clock cycle $t_p = t_3 + t_r = 100 + 10$
$= 110$ ns

→ For non-pipeline structure,

Delay time $t_n = t_1 + t_2 + t_3 + t_4 + t_r$
$= 320$ ns.

→ In this case, Speedup ratio $= \frac{320}{110}$
$= 2.9$

* Instruction Pipeline:

→ Pipeline processing can occurs not only in data stream but in instruction stream as well.

→ An instruction pipeline receives sequential instructions from memory while prior instructions are implemented in other portions.

→ Computers with complex instructions require many other phases in addition to the fetch & decode.

→ The computer needs to process each instruction with the following sequence of steps:

1. Fetch the instruction from memory (FI)
2. Decode the instruction (DA)
3. Calculate the EA
4. Fetch the operands from memory (FO)
5. Execute the instruction (EX)
6. Store the result in proper place

⇒ Example: 4-Segment Instruction Pipeline :-

→ Assume that the decoding of instruction can be combined with calculation of EA into one segment.

→ Assume that result is placed into AC so that instruction execution & storing of result can be combined into one segment.

→ (Fig. A) shows how the instruction cycle in CPU can be processed with 4-segment pipeline.

→ While an instruction is being executed in segment-4, same time next instruction in sequence is busy with fetching an operand in segment-3.

→ Similarly, an interrupt request, when detected, will cause the pipeline to empty & start again from a new address value.

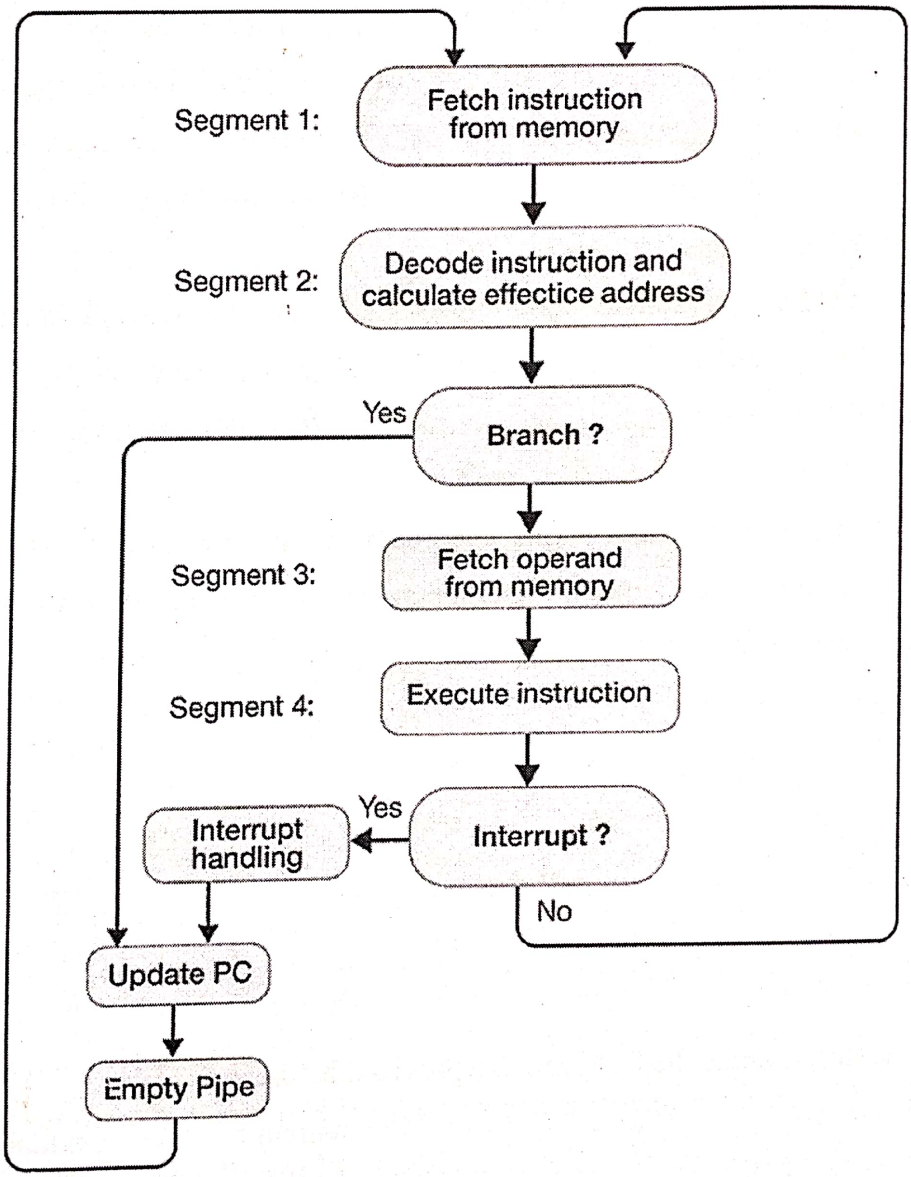


Fig. A : [4 Segment Instruction Pipeline]

→ Fig. B shows the operation of instruction pipeline.

→ Horizontal axis = Time (divided into equal steps)

→ 4-segments are represented in diagrason with symbols : FI, DA, FO, EX

Step:		1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction:	1	FI	DA	FO	EX									
	2		FI	DA	FO	EX								
(Branch)	3			FI	DA	FO	EX							
	4				FI	-	-	FI	DA	FO	EX			
	5					-	-	-	FI	DA	FO	EX		
	6									FI	DA	FO	EX	
	7										FI	DA	FO	EX

[Fig.B - Timing of Instruction Pipeline]

⇒ 1. FI - Fetches an Instruction
2. DA - Decode the instruction & Calculates effective address.
3. FO - Fetches the Operand
4. EX - Executes the instruction

⇒ In Step-4 → Instruction 1 is executed in segment EX
→ Operand for instruction 2 is being fetched in segment FO
→ Instruction 3 is being decoded in segment DA
→ Instruction 4 is being fetched in segment FI.

→ Here, we assumed that there is no branch type instruction in program.

⇒ What happened if branch instruction is present in program ?

→ Assume now that instruction 3 is a branch instruction.

→ As soon as this instruction is decoded in segment DA in step-4, transfer from FI to DA of other instructions is halted until the branch instruction is executed in step-6.

⇒ If the branch is taken, a new instruction is fetched in step-7.

→ If the branch is not taken, the instruction fetched previously in step 4 can be used.

* 3-Major difficulties in instruction pipeline :

① Resource Conflicts :
→ It caused by access to memory by two segments at same time.

② Data Dependency :-
→ This conflicts occurred, when an instruction depends on result of previous instruction, which is not yet available.

③ Branch Difficulties :
→ It arise from branch & other instructions that change the value of PC.

* RISC Pipeline :-

→ RISC instructions implement on instruction pipeline using small no. of sub-operation with each being executed in one clock cycle.

→ Because of fixed length instruction format the decoding of operation can occur at the same time at register selection.

→ All data manipulation instructions have register to register operations.

→ Since all operands are in registers, there is no need for calculating an EA or fetching of operands from memory.

→ One segment : Fetches the instruction from program memory.

→ Second segment : Executes the instruction in ALU

→ Third segment : Store the result of ALU operation in destination registers.

→ The advantage of RISC over CISC is that RISC can achieve pipeline segment requiring just one clock.

* Ex : 3 - Segment Instruction Pipeline :

→ Now considers the hardware operation for such a computers.

→ The control section fetches the instruction from memory into IR.

→ The instruction is decoded at the same time.

→ The processor unit consists of no. of registers & ALU, that performs the required arithmatic, logic & shift operations.

* 3 Segments : (1) Instruction fetch (I)

(2) ALU Operation (A)

(3) Execute instruction (E)

⇒ Consider the following 4 instructions :-

1. LOAD : R1 ← M [address 1]
2. LOAD : R2 ← M [address 2]
3. ADD : R3 ← R1 + R2
4. STORE : M [address 3] ← R3

Clock cycles:	1	2	3	4	5	6
1. Load R1	I	A	E			
2. Load R2		I	A	E		
3. Add R1 + R2			I	A	E	
4. Store R3				I	A	E

(a) Pipeline timing with data conflict

(Fig-1)

→ There will be a data conflict in instruction 3 because operand in R2 is not yet available in Segment - A. [Fig-1]

→ E segment in clock cycle 4 is in a process of placing the memory data into R2.

→ A Segment in clock cycle 4 is using the data from R2, but the value in R2 will not be the correct value, since it has not yet been ~~transferred~~ from memory.

→ If the compiler can not find useful instruction to put after load, it inserts no-op instruction. (No operation).

→ No-op is a type of instruction that is fetched from memory but has no operation. thus wasting a clock cycle.

→ This concept of delaying the use of data loaded from memory is referred to as delayed load. (Fig-2)

Clock cycle:	1	2	3	4	5	6	7
1. Load R1	I	A	E				
2. Load R2		I	A	E			
3. No-operation			I	A	E		
4. Add R1 + R2				I	A	E	
5. Store R3					I	A	E

(b) Pipeline timing with delayed load

* Examples on Pipelining :

Ex-1
QB-264 Consider a pipeline having 4 phases with duration 60 ns, 50 ns, 90 ns, 80 ns Given delay is 10 ns.

(i) Calculate Pipeline cycle time :-

Pipeline cycle time (t_p) =

Max delay due to any Stage + Register Delay

t_p = Max {60, 50, 90, 80} + 10

= 90 + 10

t_p = 100 ns

(ii) Non-pipeline execution time :

t_n = 60 + 50 + 90 + 80 + 10

= 290 ns

(iii) Speedup ratio :

$$S = \frac{t_n}{t_p} = \frac{290}{100} = 2.9$$

(iv) Total Non-pipeline time for 1000 task :

Non-pipeline system requires $n t_n$ time for completing 1000 task

$= n t_n$
$= 1000 \times 290$ ns
$= 29 \times 10^4$ ns
$= 290000$ ns
$= 290$ μs

(v) Total pipeline time for 1000 task :

$= (k + n - 1) t_p$

Here k = no. of segment = 4
$n = 1000$
$t_p = 100$ ns

Total pipeline time for 1000 task
$= (4 + 1000 - 1) 100$
$= 100300$ ns

Ex-2 3-266 A non-pipeline system takes 50ns to process a task. The same task can be processed in 6 segment pipeline with clock cycle of 10ns. Find the Speed up ratio of the pipeline for 100 task. What is the maximum Speed up that can be achieved ?

t_n = 50 ns
k = 6
t_p = 10 ns
n = 100

Speedup ratio to complete 100 task:

$$S = \frac{n\, t_n}{(k+n-1)\, t_p}$$

$$= \frac{100 \times 50}{(6+99)\,10} = 4.76$$

Maximum Speedup ratio $= S_{max} = \frac{t_n}{t_p}$

$$S_{max} = \frac{50}{10} = 5$$

Ex-3 A 4-stage pipeline has the stage delays as 110, 120, 130 and 140 ns. Registers have delay of 2ns. Find the total time taken to process 1000 instructions on this pipeline.

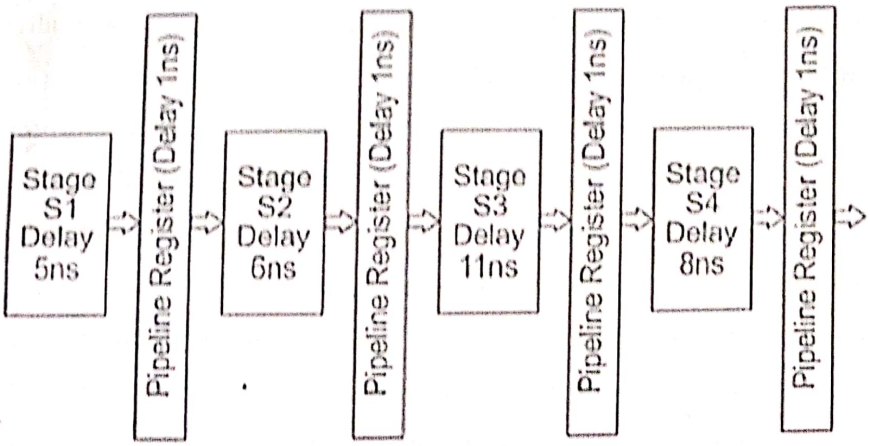
t_p = max (110, 120, 130, 140) + 2ns
= 142 ns

total pipeline time to process 1000 tasks

= (k + n − 1) t_p
= (4 + 1000 − 1) 142
= 142426 ns
= 142.426 μs

Ex-4

Consider an instruction pipeline with four stages (S1, S2, S3 and S4) each with combinational circuit only. The pipeline registers are required between each stage and at the end of the last stage. Delays for the stages and for the pipeline registers are as given in the figure.



What is the approximate speed up of the pipeline in steady state under ideal conditions when compared to the corresponding non-pipeline implementation?

$S_1 = 5$, $S_2 = 6$, $S_3 = 11$, $S_4 = 8$

Register delay = 1 ns

No. of Stage = $k = 4$

$t_p = \max(5; 6, 11, 8) + 1$

$= 12$ ns.

$t_n = 5 + 6 + 11 + 8 + 1 = 31$ ns.

$$\text{Speedup Ratio} = \frac{t_n}{t_p} = \frac{31}{12} = 2.58$$

Ex-5 Consider a pipeline having 4 phases with duration 60 ns, 50 ns, 90 ns, 80 ns Given delay is 10 ns Find out the throughput of the pipelined system. (Take $n = 1000$ tasks)

$$\text{Throughput} = \frac{\text{No. of instruction}}{\text{Total time to Complete the instruction}}$$

$$= \frac{n}{(k+n-1)t_p}$$

Here $t_p = 90 + 10 = 100$ ns

Total time to Complete 1000 tasks

$$= (k+n-1)t_p$$
$$= (4+1000-1)100$$
$$= 100300 \text{ ns}$$

$$\text{Throughput} = \frac{1000}{100300 \times 10^{-9}}$$